

Fiche 45 — Programmation dynamique déterministe et plus court chemin

Matière	Maths · Optimisation
Cours source	Bertsekas, <i>6.231 Dynamic Programming</i> , MIT OpenCourseWare, automne 2015 — cours 3
Difficulté	Fondamental — c'est le pont entre DP, graphes et optimisation combinatoire
Temps d'étude estimé	1 h 45
Prérequis	Fiche 44 (algorithme DP, principe d'optimalité), fiche 31 (flots dans les réseaux)
Concepts clés	Graphe en couches, coût-à-venir et coût-pour-arriver, DP arrière et avant, plus court chemin générique, chaînes de Markov cachées, algorithme de Viterbi, méthodes de correction d'étiquettes
Poids à l'examen	Savoir traduire un problème déterministe à états finis en plus court chemin, et savoir dire pourquoi la DP avant n'existe pas en stochastique. Le Viterbi est l'application vedette.

Vue d'ensemble

Un problème de DP **déterministe à états finis** est **exactement** un problème de plus court chemin dans un graphe en couches.

ÉTATS ↔ NŒUDS
COMMANDES ↔ ARCS
SUITES DE COMMANDES (boucle ouverte) ↔ CHEMINS de l'état initial aux états terminaux
COÛT d'une suite ↔ LONGUEUR du chemin

Cette traduction n'est pas une curiosité : elle **ouvre deux portes**. Elle donne un second algorithme, la **DP avant**, qui n'existe que dans le cas déterministe ; et elle rend disponibles **tous les algorithmes de plus court chemin** non fondés sur la DP — indispensables dès que l'espace d'états devient énorme.

● Concept 1 — Le dictionnaire graphe / DP

On considère un problème déterministe à états finis : à l'instant k , l'état vit dans un ensemble fini S_k .

Notation	Sens
a_{ij}^k	coût de transition de l'état $i \in S_k$ vers l'état $j \in S_{k+1}$ à l'instant k — vu comme la longueur de l'arc
a_{it}^N	coût terminal de l'état $i \in S_N$ — l'arc vers le nœud terminal artificiel t
s	l'état initial

Le graphe est **en couches** : étape 0, étape 1, ..., étape N , plus un nœud terminal artificiel t relié à tous les états finaux par des **arcs terminaux de longueur égale au coût terminal**.

POURQUOI LE NŒUD ARTIFICIEL t .

Il transforme « minimiser le coût sur toutes les trajectoires, avec un coût terminal qui dépend de l'état d'arrivée » en « trouver le plus court chemin **entre deux nœuds fixés** s et t ». On revient au problème de graphe le plus standard qui soit.

● Concept 2 — DP arrière et DP avant

Algorithme DP (arrière).

$$J_N(i) = a_{it}^N, \quad i \in S_N$$

$$J_k(i) = \min_{j \in S_{k+1}} \left[a_{ij}^k + J_{k+1}(j) \right], \quad i \in S_k, \quad k = 0, \dots, N-1$$

Le coût optimal est $J_0(s)$, et il est égal à la **longueur du plus court chemin de s à t** .

L'observation qui donne l'algorithme avant.

*Un chemin optimal $s \rightarrow t$ est aussi un chemin optimal $t \rightarrow s$ dans le problème de plus court chemin **inversé**, où la direction de chaque arc est renversée et sa longueur laissée inchangée.*

Algorithme DP avant — c'est la DP arrière appliquée au problème inversé :

$$\tilde{J}_N(j) = a_{sj}^0, \quad j \in S_1$$

$$\tilde{J}_k(j) = \min_{i \in S_{N-k}} \left[a_{ij}^{N-k} + \tilde{J}_{k+1}(i) \right], \quad j \in S_{N-k+1}$$

et le coût optimal est

$$\tilde{J}_0(t) = \min_{i \in S_N} [a_{it}^N + \tilde{J}_1(i)]$$

L'interprétation, essentielle. On voit $\tilde{J}_k(j)$ comme le **coût-pour-arriver optimal** à l'état j depuis l'état initial s — par symétrie avec le **coût-à-venir** $J_k(i)$ de la DP arrière.

	DP arrière	DP avant
Quantité calculée	coût-à-venir : de i jusqu'à t	coût-pour-arriver : de s jusqu'à j
Sens de parcours	de N vers 0	de 0 vers N
Existe en stochastique ?	oui	non

● Concept 3 — Pourquoi il n'y a pas de DP avant en stochastique

C'est l'un des points les plus profonds du cours, et il tient en trois arguments.

Il n'existe pas d'algorithme DP avant pour les problèmes stochastiques.

Mathématiquement : en stochastique, on ne peut pas se restreindre aux **suites en boucle ouverte** (fiche 44) ; or c'est précisément l'identification « suite de commandes $\leftrightarrow$ chemin » qui fonde le point de vue plus court chemin. Celui-ci **s'effondre**.

Conceptuellement : en présence d'incertitude, la notion de **coût-pour-arriver optimal** à un état x_k **n'a pas de sens**. Il peut par exemple être **impossible de garantir**, avec probabilité **1**, qu'un état donné sera atteint.

Par contraste : même en stochastique, la notion de **coût-à-venir optimal** depuis un état x_k garde un sens parfaitement clair.

La conséquence pratique. Toute méthode fondée sur un « coût-pour-arriver » — beaucoup d'algorithmes de graphe, dont A^* — est cantonnée au **déterministe**. Dès qu'il y a de l'aléa, il ne reste que la récurrence à rebours. C'est une **asymétrie fondamentale** du temps en décision stochastique : on peut raisonner sur l'avenir en espérance, pas sur le passé qui aurait pu être.

● Concept 4 — Le problème de plus court chemin générique

Données. Des nœuds $\{1, 2, \dots, N, t\}$ (t étant la **destination**), et a_{ij} le coût du déplacement du nœud i au nœud j . On cherche un plus court chemin de **chaque** nœud i vers t .

Hypothèse. *Tous les cycles sont de longueur positive ou nulle.* Alors **un chemin optimal n'a pas besoin de plus de N déplacements** — repasser par un nœud ne peut qu'ajouter la longueur d'un cycle, donc ne fait jamais gagner.

La mise en forme DP. On impose **exactement N déplacements**, en autorisant des déplacements **dégénérés** d'un nœud i vers lui-même, de coût $a_{ii} = 0$. On pose

$$J_k(i) = \text{coût optimal pour aller de } i \text{ à } t \text{ en } N - k \text{ déplacements}$$

si bien que $J_0(i)$ est le coût du chemin optimal de i à t .

Algorithme.

$$J_{N-1}(i) = a_{it}, \quad i = 1, \dots, N$$

$$J_k(i) = \min_{j=1, \dots, N} [a_{ij} + J_{k+1}(j)], \quad k = 0, 1, \dots, N-2$$

L'astuce des déplacements dégénérés mérite d'être retenue : elle transforme « au plus N étapes » en « exactement N étapes », donc un problème à horizon **variable** en un problème à horizon **fixe** — le seul que sache traiter l'algorithme DP standard.

● Concept 5 — Chaînes de Markov cachées et algorithme de Viterbi

C'est l'application la plus célèbre du plus court chemin, et elle est spectaculaire.

Le cadre. Une chaîne de Markov de probabilités de transition p_{ij} , dont les **transitions d'état sont cachées**. À chaque transition, on obtient une **observation indépendante** : $r(z; i, j)$ est la probabilité que l'observation vaille z lorsque la transition va de i à j .

Le problème d'estimation de trajectoire. Étant donné la suite d'observations $Z_N = \{z_1, \dots, z_N\}$, quelle est la suite de transitions d'états **la plus vraisemblable** $\hat{X}_N = \{\hat{x}_0, \dots, \hat{x}_N\}$, c'est-à-dire celle qui maximise $p(X_N | Z_N)$ sur toutes les suites X_N ?

La réduction (l'algorithme de Viterbi).

Étape 1 — se débarrasser du conditionnement.

$$p(X_N | Z_N) = \frac{p(X_N, Z_N)}{p(Z_N)}$$

Le dénominateur **ne dépend pas de X_N** : maximiser $p(X_N | Z_N)$ équivaut à maximiser $p(X_N, Z_N)$, donc $\ln p(X_N, Z_N)$.

Étape 2 — *factoriser* par la règle de multiplication des probabilités conditionnelles :

$$p(X_N, Z_N) = \pi_{x_0} \prod_{k=1}^N p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k)$$

où π_{x_0} est la loi initiale.

Étape 3 — *passer au logarithme*, ce qui transforme le produit en somme et le maximum en minimum :

$$\min_{\{x_0, \dots, x_N\}} \left(-\ln \pi_{x_0} - \sum_{k=1}^N \ln (p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k)) \right)$$

C'est un problème de plus court chemin. Les nœuds sont les états à chaque instant, la longueur de l'arc de x_{k-1} vers x_k à l'étape k est $-\ln (p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k))$, et le coût de départ est $-\ln \pi_{x_0}$. On le résout par la DP.

Pourquoi le logarithme est le geste décisif. Il convertit un **produit** de probabilités — qui est multiplicatif, donc étranger à la structure additive de la DP — en une **somme**. C'est ce qui rend le problème *additif dans le temps*, condition d'application de la DP (fiche 44). Et comme $-\ln$ est décroissante, maximiser la vraisemblance devient minimiser une longueur : les probabilités deviennent des **distances**.

Les longueurs $-\ln(pr)$ sont **positives** (des probabilités sont ≤ 1), ce qui garantit l'hypothèse « cycles de longueur positive » du concept 4.

● Concept 6 — Algorithmes de plus court chemin non fondés sur la DP

Il existe beaucoup d'algorithmes de plus court chemin qui ne relèvent pas de la DP. Tous peuvent servir à résoudre des problèmes déterministes à états finis. Ils peuvent être **préférables** à la DP s'ils évitent de calculer le coût-à-venir optimal de **CHAQUE** état.

C'est l'argument décisif pour les très grands espaces d'états. La DP calcule $J_k(i)$ pour **tous** les i et **tous** les k : c'est exhaustif, donc rédhibitoire quand l'espace d'états est immense. Un algorithme qui explore seulement les nœuds prometteurs peut aboutir sans jamais visiter le reste.

L'exemple emblématique : l'optimisation combinatoire — ordonnancement, voyageur de commerce. Le cours illustre par un arbre d'états où chaque nœud est une **séquence partielle** de villes visitées (A , AB , ABC , $ABCD$...), les arcs portant les distances, et un nœud terminal artificiel t refermant les séquences complètes.

Les méthodes de correction d'étiquettes

Données. Une origine s , une destination t , des longueurs $a_{ij} \geq 0$. **L'idée.** Découvrir **progressivement** des chemins de plus en plus courts de s vers chaque nœud i .

Notations.

Symbole	Sens
d_i (étiquette de i)	longueur du plus court chemin trouvé jusqu'ici ; initialement $d_s = 0$ et $d_i = \infty$ pour $i \neq s$
UPPER	l'étiquette d_t de la destination
liste OPEN	les nœuds actifs , candidats à un examen ultérieur ; initialement OPEN = $\{s\}$

L'algorithme.

Étape 1 (retrait d'un nœud). Retirer un nœud i de OPEN et, pour chaque enfant j de i , effectuer l'étape 2.

Étape 2 (test d'insertion). Si $d_i + a_{ij} < \min\{d_j, \text{UPPER}\}$ alors poser $d_j := d_i + a_{ij}$ et déclarer i parent de j ;
de plus, si $j \neq t$, placer j dans OPEN s'il n'y est pas déjà ;
si $j = t$, poser $\text{UPPER} := d_i + a_{it}$.

Étape 3 (test d'arrêt). Si OPEN est vide, terminer ; sinon retourner à l'étape 1.

Les deux questions que pose le test d'insertion — c'est la façon de le mémoriser :

$d_i + a_{ij} < \text{UPPER}$? « le chemin $s \rightarrow i \rightarrow j$ a-t-il une CHANCE de faire partie d'un $s \rightarrow t$ plus court ? »

$d_i + a_{ij} < d_j$? « le chemin $s \rightarrow i \rightarrow j$ est-il MEILLEUR que le $s \rightarrow j$ courant ? »

Le premier test est un élagage exactement de la même nature que celui du branch-and-bound (fiche 33) : `UPPER` joue le rôle de l'incumbent, et tout chemin partiel qui dépasse déjà cette borne est abandonné **sans être exploré**. Le second test évite de refaire un travail déjà fait.

L'ordre dans lequel on retire les nœuds de `OPEN` n'est **pas** spécifié — et c'est là que se logent toutes les variantes : le retirer selon d_i minimal donne **Dijkstra**, en file donne **Bellman-Ford**, en pile donne une exploration en profondeur.

Comment résoudre l'exercice type (protocole)

1. **Vérifier que le problème est déterministe et à états finis** — sinon, pas de traduction en graphe.
2. **Construire le graphe en couches** : un nœud par couple (état, instant), un arc par transition possible, la longueur étant le coût de transition.
3. **Ajouter le nœud terminal artificiel t** , relié aux états finaux par des arcs de longueur égale au coût terminal.
4. **Choisir le sens** : DP arrière si l'on veut le coût-à-venir depuis tout état ; DP avant si l'on veut le coût-pour-arriver.
5. **Dérouler la récurrence** en enregistrant à chaque nœud le coût optimal **et** l'arc choisi.
6. **Reconstituer le chemin optimal** en remontant les parents depuis t .
7. **Si l'espace d'états est énorme** : renoncer à la DP exhaustive et passer à une méthode de correction d'étiquettes, en soignant l'ordre de retrait de `OPEN`.

Exercices progressifs

Niveau 1 — Pourquoi ajoute-t-on un nœud terminal artificiel t ?

Parce que le coût terminal $g_N(x_N)$ dépend de l'état d'arrivée : sans nœud unique, on aurait « le plus court chemin de s vers **l'un quelconque** des états finaux, avec une pénalité par état ». En reliant chaque état final $i \in S_N$ à un nœud unique t par un arc de longueur $a_{it}^N = g_N(i)$, on obtient un problème de plus court chemin **entre deux nœuds fixés** — la forme canonique.

Niveau 2 — Justifiez l'hypothèse « tous les cycles sont de longueur positive ou nulle » dans le problème générique.

Elle garantit qu'un chemin optimal ne repasse jamais par un nœud, donc utilise au plus N déplacements. En effet, un chemin qui repasserait par un nœud contient un cycle ; le supprimer retranche la longueur de ce cycle, qui est ≥ 0 : le chemin raccourci est donc au moins aussi court.

Si l'hypothèse tombe — s'il existe un cycle de longueur **strictement négative** — le problème n'a pas de solution : on peut parcourir ce cycle indéfiniment et faire tendre la longueur vers $-\infty$. C'est l'analogue exact d'un LP non borné (fiche 24).

Niveau 3 — Expliquez pourquoi l'algorithme de Viterbi passe par le logarithme.

Le critère naturel est $p(X_N, Z_N) = \pi_{x_0} \prod_k p_{x_{k-1}x_k} r(z_k; \cdot)$: c'est un **produit** le long de la trajectoire. Or la DP exige un coût **additif** dans le temps (fiche 44) — c'est l'hypothèse structurelle de tout le formalisme.

Le logarithme convertit le produit en **somme** :

$$\ln p(X_N, Z_N) = \ln \pi_{x_0} + \sum_k \ln (p_{x_{k-1}x_k} r(z_k; \cdot))$$

et comme **ln** est **strictement croissante**, maximiser p équivaut à maximiser **ln p**, donc à **minimiser** $-\ln p$ — une somme de termes positifs, c'est-à-dire une **longueur de chemin**.

Le geste général à retenir : face à un critère multiplicatif, passer au logarithme pour retrouver l'additivité. C'est le même mouvement qui rend convexe un programme géométrique (fiche 37).

Niveau 4 — type examen — Sur le graphe à quatre nœuds $\{1, 2, 3, t\}$ avec $a_{12} = 2$, $a_{13} = 5$, $a_{23} = 1$, $a_{2t} = 7$, $a_{3t} = 3$, et $a_{ij} = \infty$ sinon, calculez $J_0(1)$ par la récurrence du concept 4 avec $N = 3$.

On autorise les déplacements dégénérés $a_{ii} = 0$.

Initialisation ($k = N - 1 = 2$) : $J_2(i) = a_{it}$, soit

$$J_2(1) = \infty, \quad J_2(2) = 7, \quad J_2(3) = 3$$

Étape $k = 1$: $J_1(i) = \min_j [a_{ij} + J_2(j)]$, le déplacement dégénéré $j = i$ donnant $0 + J_2(i)$.

$$J_1(1) = \min \left\{ \underbrace{0 + \infty}_{j=1}, \underbrace{2 + 7}_{j=2}, \underbrace{5 + 3}_{j=3} \right\} = 8$$

$$J_1(2) = \min\{0 + 7, 1 + 3\} = 4, \quad J_1(3) = \min\{0 + 3\} = 3$$

Étape $k = 0$:

$$J_0(1) = \min\{0 + 8, 2 + 4, 5 + 3\} = 6$$

$$J_0(2) = \min\{0 + 4, 1 + 3\} = 4, \quad J_0(3) = 3$$

Conclusion. $J_0(1) = 6$, réalisé par $1 \rightarrow 2$ (coût 2) puis le chemin optimal depuis 2, qui est $2 \rightarrow 3 \rightarrow t$ (coût $1 + 3 = 4$). Le plus court chemin est donc $1 \rightarrow 2 \rightarrow 3 \rightarrow t$, de longueur 6.

Ce que l'exercice illustre. Le chemin direct $1 \rightarrow 3 \rightarrow t$ coûte $5 + 3 = 8$, et $1 \rightarrow 2 \rightarrow t$ coûte $2 + 7 = 9$: c'est la combinaison $1 \rightarrow 2 \rightarrow 3 \rightarrow t$ qui gagne. Aucune décision **myope** (regarder seulement le prochain arc) ne l'aurait trouvée — depuis 1, l'arc le moins cher mène à 2, mais depuis 2 la suite naturelle serait t . Seule la récurrence, qui **propage l'information depuis la destination**, découvre le bon chemin. C'est précisément ce que la DP apporte.

● Common mistakes

1. **Chercher un algorithme DP avant en stochastique** — il n'en existe pas ; le coût-pour-arriver n'a pas de sens sous incertitude.
2. **Oublier le nœud terminal artificiel** — sans lui, le coût terminal n'est pas intégré au graphe.
3. **Oublier les déplacements dégénérés $a_{ii} = 0$** dans le problème générique — sans eux, on impose de faire **exactement** N vrais déplacements.
4. **Négliger l'hypothèse sur les cycles** — avec un cycle de longueur négative, le problème est non borné.
5. **Appliquer la DP sur un critère multiplicatif** — il faut d'abord passer au logarithme (Viterbi).
6. **Utiliser la DP exhaustive sur un espace d'états énorme** — elle calcule le coût-à-venir de **chaque** état ; une méthode d'étiquettes peut aboutir sans les visiter tous.
7. **Confondre les deux tests d'insertion** — l'un compare à `UPPER` (élagage global), l'autre à d_j (amélioration locale). Les deux sont nécessaires.
8. **Oublier d'enregistrer le parent** — sans lui, on obtient la **longueur** du plus court chemin, pas le chemin.

Ultimate Review

1. **Dictionnaire** : états $\leftrightarrow$ nœuds, commandes $\leftrightarrow$ arcs, suites en boucle ouverte $\leftrightarrow$ chemins, coût $\leftrightarrow$ longueur.
2. **DP arrière** : $J_N(i) = a_{it}^N$, $J_k(i) = \min_j [a_{ij}^k + J_{k+1}(j)]$; coût optimal $J_0(s)$.
3. Un chemin optimal $s \rightarrow t$ est optimal $t \rightarrow s$ dans le graphe **inversé** — d'où la **DP avant**.
4. **Coût-à-venir** (arrière) contre **coût-pour-arriver** (avant).
5. **Pas de DP avant en stochastique** : on ne peut pas se restreindre aux suites en boucle ouverte, et le coût-pour-arriver n'a pas de sens ; le coût-à-venir, lui, en a toujours un.
6. **Plus court chemin générique** : cycles de longueur ≥ 0 , au plus N déplacements, déplacements dégénérés $a_{ii} = 0$, $J_{N-1}(i) = a_{it}$ puis $J_k(i) = \min_j [a_{ij} + J_{k+1}(j)]$.
7. **Viterbi** : maximiser $p(X_N | Z_N) \iff$ maximiser $p(X_N, Z_N) \iff$ minimiser $-\ln \pi_{x_0} - \sum_k \ln(p r)$ — un plus court chemin.
8. Les algorithmes non-DP sont **préférables** quand ils évitent de calculer le coût-à-venir de **chaque** état — indispensable en optimisation combinatoire.
9. **Correction d'étiquettes** : étiquettes d_i , borne **UPPER**, liste **OPEN**, deux tests — « a-t-il une chance ? » et « est-il meilleur ? ».

Formulas to know

$$J_N(i) = a_{it}^N, \quad J_k(i) = \min_{j \in S_{k+1}} [a_{ij}^k + J_{k+1}(j)]$$

$$\tilde{J}_0(t) = \min_{i \in S_N} [a_{it}^N + \tilde{J}_1(i)] \quad \min \left(-\ln \pi_{x_0} - \sum_{k=1}^N \ln (p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k)) \right)$$

$$d_i + a_{ij} < \min\{d_j, \text{UPPER}\}$$

Methods to know : le protocole en 7 étapes ; la construction du graphe en couches ; la réduction de Viterbi ; l'algorithme de correction d'étiquettes.

Active Recall

Basic — Donnez le dictionnaire entre DP déterministe à états finis et plus court chemin.

États $\leftrightarrow$ nœuds ; commandes $\leftrightarrow$ arcs ; suites de commandes en boucle ouverte $\leftrightarrow$ chemins de l'état initial aux états terminaux ; coût d'une suite $\leftrightarrow$ longueur du chemin. Le coût de transition a_{ij}^k est la longueur de l'arc, et le coût terminal a_{it}^N celle de l'arc vers le nœud terminal artificiel t .

Understanding — Pourquoi n'y a-t-il pas de DP avant en stochastique ?

Mathématiquement : en stochastique on ne peut pas se restreindre aux suites en boucle ouverte, or c'est l'identification « suite $\leftrightarrow$ chemin » qui fonde le point de vue plus court chemin ; il s'effondre. **Conceptuellement** : le « coût-pour-arriver optimal » à un état x_k n'a pas de sens sous incertitude — il peut être impossible de garantir avec probabilité 1 qu'un état sera atteint. Le « coût-à-venir » depuis x_k , en revanche, garde toujours un sens clair.

Application — Comment traduit-on « au plus N déplacements » en un problème DP à horizon fixe ?

En imposant **exactement** N déplacements et en autorisant des déplacements **dégénérés** d'un nœud vers lui-même, de coût $a_{ii} = 0$. Un chemin utilisant $m < N$ vrais déplacements se complète par $N - m$ déplacements dégénérés sans changer sa longueur. L'hypothèse « cycles de longueur ≥ 0 » garantit qu'aucun chemin optimal n'a besoin de plus de N déplacements.

Comparison — DP exhaustive et méthodes de correction d'étiquettes : quand préférer l'une ou l'autre ?

La **DP** calcule le coût-à-venir optimal de **chaque** état à chaque instant : elle donne donc la politique complète, mais son coût croît avec la taille de l'espace d'états. Les méthodes d'**étiquettes** ne visitent que les nœuds prometteurs, grâce à l'élagage par **UPPER** : elles sont préférables quand l'espace d'états est **énorme** et qu'on ne veut qu'**un** chemin optimal depuis une origine donnée — cas typique de l'optimisation combinatoire (ordonnancement, voyageur de commerce).

Exam-style — Décrivez l'algorithme de Viterbi et dites quel est son geste décisif.

Cadre. Chaîne de Markov de transitions p_{ij} cachées, observations indépendantes de loi $r(z; i, j)$. On cherche la trajectoire $\hat{X}_N$ maximisant $p(X_N | Z_N)$.

Réduction. Comme $p(Z_N)$ ne dépend pas de X_N , maximiser $p(X_N | Z_N)$ revient à maximiser $p(X_N, Z_N) = \pi_{x_0} \prod_k p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k)$, donc à minimiser

$$-\ln \pi_{x_0} - \sum_{k=1}^N \ln (p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k))$$

C'est un **plus court chemin** dans un graphe en couches dont les longueurs d'arcs sont ces $-\ln$, positives.

Le geste décisif : le logarithme. Il transforme un critère **multiplicatif** — étranger à la DP — en un critère **additif**, seule structure que sache exploiter la récurrence de Bellman.

Flashcards

Question	Réponse
DP déterministe à états finis = ?	Un problème de plus court chemin dans un graphe en couches
À quoi correspondent les commandes ?	Aux arcs du graphe
Rôle du nœud terminal artificiel t ?	Intégrer le coût terminal comme longueur d'arc
Récurrance DP arrière ?	$J_k(i) = \min_j [a_{ij}^k + J_{k+1}(j)]$, $J_N(i) = a_{it}^N$
Que calcule la DP arrière ?	Le coût-à-venir de i jusqu'à t
Que calcule la DP avant ?	Le coût-pour-arriver de s jusqu'à j
D'où vient la DP avant ?	C'est la DP arrière sur le graphe inversé
Existe-t-il une DP avant en stochastique ?	Non — le coût-pour-arriver n'y a pas de sens
Hypothèse du plus court chemin générique ?	Tous les cycles de longueur ≥ 0
Astuce des déplacements dégénérés ?	$a_{ii} = 0$, pour passer d'« au plus N » à « exactement N »
Problème résolu par Viterbi ?	Trajectoire la plus vraisemblable d'une chaîne de Markov cachée
Le geste clé de Viterbi ?	Le logarithme : produit $\rightarrow$ somme, donc coût additif
Longueur d'arc dans Viterbi ?	$-\ln(p_{x_{k-1}x_k} r(z_k; x_{k-1}, x_k))$
Quand préférer un algorithme non-DP ?	Quand il évite de calculer le coût-à-venir de chaque état
Les trois objets des méthodes d'étiquettes ?	Étiquettes d_i , borne UPPER , liste OPEN
Le test d'insertion ?	$d_i + a_{ij} < \min\{d_j, \text{UPPER}\}$
Que change l'ordre de retrait de OPEN ?	Dijkstra, Bellman-Ford, exploration en profondeur